RouterAcc: FPGA Acceleration for VLSI Detailed Router via Hierarchical Storage Mapping

***Abstract*—The semiconductor industry keeps seeking for reducing the design time and efforts in modern integrated circuit implementation, while detailed routing is a dead-or-alive critical element in design automation tooling for advanced node enablement, which is known as the most time-consuming step in the back-end design flow due to its high complexity and enormous solution space for the very large-scale integration (VLSI) routing problem.**

In this paper, we propose a FPGA-based implementation to accelerate the detailed routers, uses efficient memory access management and parallelism to achieve better performance. Experimental results demonstrate that detailed routers obtain stable and effective acceleration on FPGA. Our design gets 2.1×-2.3×, 2.1×-2.5×, 1.3× speedup compared to TritonRoute, Dr.CU,and current optimum routing kernel acceleration work, also obtains 20% speedup for detailed routing flow, and 1.0×-1.2× resource utilization compared to TritonRoute.

***Index Terms*—Reconfigurable Computing, FPGA Acceleration, VLSI, Detailed Router**

I. Introduction

Routing is known as the most complex and time-consuming stage in the physical design flow due to its high complexity and enormous solution space for the very large-scale integration (VLSI) routing problem. Routing is categorized into global routing and detailed routing. Global routing provides rough guidance with congestion and timing. Detailed routing generates exact rectilinear wiring interconnects based on global routing results and reduces the design rule violations to satisfy the manufacturability requirements. Detailed routing is the primary time bottleneck compared to global routing due to strict DRC adherence, larger search space, dynamic data management, manufacturing variability handling, and toolchain interoperability issues.

Detailed routers like TritonRoute[**?**] and Dr.CU 2.0[**?**] have demonstrated promising performance on industrial benchmarks from ISPD 2018/2019 contests, are widely accepted in industry and academia to date. TritonRoute, as the CPU-driven routing engine within the state-of-the-art open-source RTL-to-GDSII flow, delivers design-rule-clean routing solutions for advanced technology nodes at 16nm and below. While this methodology demonstrates robust capability in resolving complex multi-constrained routing problems with high solution quality, its inherent computational inefficiency presents a significant bottleneck that markedly impacts turnaround time.

To accelerate routing, existing approaches primarily focus on utilizing partition-based multithreaded CPUs or GPU-accelerated architectures. However, these methods exhibit diminishing returns as thread counts increase beyond saturation points, often resulting in measurable quality degradation. By customizability of field-programmable gate array (FPGA), the realization of hardware and software co-acceleration with FPGA has already achieved competitive performance in accelerating computation tasks in fields like machine learning and scientific computing. Similarly, we can benefit from FPGA acceleration in the back-end algorithms by customizing and optimizing specific algorithms and detailed routing flow to provide higher performance and faster speed.

In this work, we propose RouterAcc, a highly parallel iterative FPGA acceleration framework integrated with multi-dimensional hierarchical storage mapping for Detailed Router. The key contributions are summarized as follows:

- A memory access management-enabled FPGA acceleration for detailed router is proposed.Instantiate multiple ram groups to store node information on the corresponding dimension, and Provides a flexible dimension partition architecture, RAM storage can be divided according to specific data characteristics according to X, Y, Z dimensions to maximize resource utilization.
- We propose efficient FPGA implementations of detailed routing Kernel on FPGA, and implement detailed routing methods based on grid and A* in workers.
- We demonstrate 2.1×-2.3× speedup in detailed routing without quality degradation of the entire flow over multi-threaded FPGA kernels implementations.More specifically,search operations are independent of each other so that the framework maintains nearly linear scalability with industrial designs.
- Through the trial of TritonRoute and Dr.CU accelerating, it is proved that the proposed optimization method has universal significance in the detailed routing scheme, and can be transferred to other similar work besides the example studied in this paper.

II. Preliminaries

A. Detailed Routing Acceleration

Routing is a complex and critical stage in the physical design of Very Large Scale Integration(VLSI), directly impacting chip performance, power consumption, and manufacturing costs. The detailed routing algorithm is critical to routing closure. It generates exact rectilinear wiring interconnects based on global routing results and reduces the design rule violations to satisfy the manufacturability requirements.

In the previous work, BonnRoute[**?**] and RegularRoute[**?**] have made some breakthroughs in reducing wire lengths/through holes and ensuring design compliance

through global resource optimization and rule-based hierarchical routing strategies. Triggered by the ISPD'18[**?**] and ISPD'19[**?**] detailed routing contest, many novel routing methodologies have emerged. Kahng et al.[**?**]partitioned each layer into parallel panels and formulated the routing problem on each panel as an integer linear program. Dr. CU[**?**] proposed an optimal path searching algorithm for minimum-area constraints. Based on this, Li et al. proposed Dr. CU 2.0[**?**]to improve the routing quality. TritonRoute[**?**] employs the rip-up and reroute (RRR) scheme to optimize routing quality and use a routing kernel to find the path of each net. Among these algorithms, TritonRoute has the best solution quality and the lowest level of design rule violations (DRCs). However, repeated operations on large-scale nets consume most of the total runtime by the routing kernel, establishing it as the dominant efficiency bottleneck in detailed routing flows.

To address the critical challenge of time-consuming routing processes in routing flow, researchers have explored heterogeneous acceleration platforms. For path planning acceleration using the A* algorithm, Zhou et al.[**?**] demonstrated 37-75× acceleration via 8-port cache and array-based OPEN lists. Osama et al.[**?**] developed an FPGA implementation with 50-100× speedup through 8-way parallel evaluators and cache-optimized memory access. However, none of these methods have been applied to the detailed routing kernel. For maze routing acceleration, Lin et al.[**?**] proposed GAMER that achieved 16× acceleration for coarsened maze routing through sweeping-style parallelism. Jiang et al.[**?**] implemented an FPGA-based batched maze routing kernel, achieving 3.1× speedup over CPU implementations with ¡1% wirelength degradation. However, the performance of these methods on detailed routing is constrained by data dependencies and memory management.

B. Memory Access Bottleneck of Detailed Routing

The core of detailed routing algorithms lies in the highly efficient heuristic path-planning method, like the A* search algorithm. However, the existing optimizations on the A* algorithm are mainly aimed at computational optimization, and the bottleneck in maintaining the open list and the closed list is outstanding, especially when performing many iterative operations.

Zhou et al.[**?**] found that when the A* algorithm runs in a single thread, the operations of the open list consume 95% of the total time. After distributing the software process into eight threads, the open list operations still consume 88% of total time. Thus the architecture does not solve the problem of time-consuming memory access, and suffers from inefficient resource utilization when processing high-obstacle density maps and frequent cache misses degrade performance predictability. Therefore, the most time-consuming open list and close list operations have become the bottleneck of A* algorithm acceleration. The accelerated A* techniques exhibit broad applicability across various path-searching implementations. Given the fundamental similarities in routing paradigms, particularly in maze routing algorithms and heuristic flow implementation methodologies, these acceleration strategies demonstrate significant transfer potential beyond the specific use cases examined in this study.

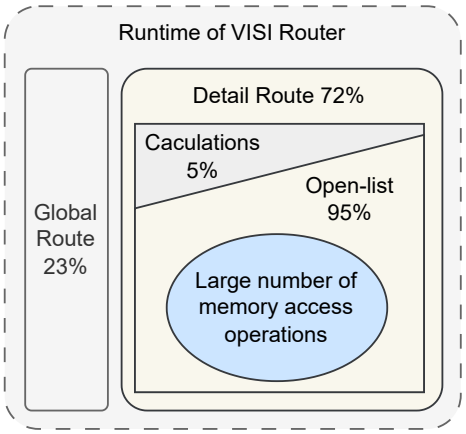


Fig. 1. Detailed routing takes up most time in the routing process, and most of it is memory access time.

Osama et al.[8] developed an FPGA implementation with 50-1001× speedup through 8-way parallel evaluators and cache-optimized memory access. Nevertheless, its fixed 256×256 grid mapping and complex 5×5 caching mechanism restrict scalability for large-scale scenarios. Jiang et al.[9] implemented an FPGA-based batched maze routing kernel achieving 3.11× speedup over CPU implementations with ¡ 1% wirelength degradation, but its performance is constrained by data dependency management and limited scalability when handling heterogeneous net sizes. Meanwhile, Lin et al.[10] proposed GAMER, a GPU-accelerated maze routing scheme that achieved 161× acceleration for coarsened maze routing through sweeping-style parallelism. However, its performance gain diminishes in fine-grained routing stages, and the qualitydriven batch partitioning strategy introduces additional scheduling overhead that limits throughput.

C. Versatility and Scalability

The above proposed acceleration methodology for detailed routing preserves the quality of the entire implementation flow for multithreaded FPGA core architectures. Actually, the accelerated A* techniques exhibit broad applicability across various path-searching implementations. Given the fundamental similarities in routing paradigms, particularly in maze routing algorithms and heuristic flow implementation methodologies, these acceleration strategies demonstrate significant transfer potential beyond the specific use cases examined in this study.

III. ALGORTHM

A. Detailed Routing Algorthm Analysis

Conventional framework of detailed routing algorithm we employ follows a structured flow, as illustrated in Algorithm 1. Given the global routing results of the benchmarks (e.g..def and.guides files), we perform a track assignment step following

the method described in [11], aligning all nets to predefined routing tracks. This preprocessing phase extracts and constructs essential data structures, such as routing grids, net objects, and guide constraints, which serve as inputs to the detailed routing engine. The ultimate goal is to produce a DRC-clean routing result with full connectivity, starting from these coarse global routing guides.

Algorithm 1 Detailed Routing Flow

Input: base-data structure, global routing guides
Output: full connected nets

1: InitDb(), InitPreRouteNet()
2: **while** $iter < maxIter$ **do**
3: $drcNets \leftarrow [net_i : while\ net_i \in Nets, if\ net_i.drcScore \neq 0]$
4: **while** $net \in drcNets$ **do**
5: $ucPins \leftarrow GetPins(net)$
6: **while** $ucPins \neq \emptyset$ **do**
7: $tpin \leftarrow \underset{pin \in ucPins}{\arg\max}\ pin.cost$
8: $cPins \leftarrow cPins \cup tPin,\ ucPins \leftarrow ucPins \setminus tPin$
9: $sPin \leftarrow$ GetNextPin($ucPins$)
10: FPGA-OptimizedRoutingKernel($sPin$,$cPins$)
11: **end while**
12: **end while**
13: UpdateDRC(), UpdateNets()
14: **end while**
15: UpdateDb()

The detailed routing flow, as depicted in Algorithm 1, comprises multiple sequential phases. Initially, the design database is initialized via InitDb(), and essential nets are pre-routed using InitPreRouteNet(). Following this, nets are evaluated based on their design rule check scores, and those exhibiting violations are prioritized for further processing. The outer loop (lines 2 to 14) performs iterative rip-up and reroute operations, bounded by a predefined maximum iteration count. In each iteration, the set of nets with non-zero DRC scores is extracted. For each net in drcNets, the set of unconnected pins is identified (line 5). The algorithm proceeds to incrementally connect these pins by repeatedly selecting the pin with the highest routing cost as the current target, moving it to the connected pin set, and identifying the next nearest source pin (lines 6–9). Routing between the selected source and target pins uses the Optimized Routing kernel, which leverages high-throughput hardware acceleration. This process continues until all pins in the net are connected. After net-level rerouting, the global DRC state and net data structures are updated accordingly . Upon convergence or exhaustion of iterations, a final database update is performed, reflecting the complete routing solution.

B. Routing Kernel and Generating Routing Graphs Algorthm Analysis

The most popular detailed routing methods, like TritonRoute[?] and Dr. CU[?], are both executed by the maze routing kernel to connect all pins on the nets with minimal cost. In the actual computing stage, the nets will first abstracted as grid graphs. Each pin pn is equivalent to a set of vertices Sn located on its position, then utilize the priority queue and some heuristic methods to improve the quality and speed.Taking TritonRoute[?] as an example, it uses A* (A-STAR) algorithm to explore the route and sets weight for each node. The weight is a heuristic function f (x) : f (x) = g(x) + h(x), where the function g(x) is the distance or cost from the starting node to current state x, and the function h(x) defines the estimated distance or cost from current state x to the end node. We call the function value of f (x) the f value.

Algorithm 2 FPGA-Optimized Routing Kernel Algorithm

Input: source pin s, target pins $\{t_1, \ldots, t_n\}$
Output: minimum-cost routing paths
Prepare: priority queue Q, cost map T, flag map F, graph batch $G\ \{G_1, ..., G_N\}$, direction set $D\ \{E, W, N, S, U, D\}$ in BRAM

1: Initialize N grid maps in BRAM
2: **for** $G_k \in G$ **in parallel do**
3: **set** $Q^{(k)}$, $T^{(k)}(v) \leftarrow \infty$, $F^{(k)}(v) \leftarrow 0, \forall v$; $T^{(k)}(s) \leftarrow 0$
4: **while** $Q^{(k)} \neq \emptyset$ **do**
5: $v \leftarrow$ MinCost($Q^{(k)}$)
6: $if\ v \in \{t_1, ..., t_n\}$ **break**
7: **for** $u \in D$ **in parallel do**
8: $c \leftarrow T^{(k)}(v) + \Delta(v, u)$
9: **if** $c < T^{(k)}(u)$ **then**
10: $T^{(k)}(u) \leftarrow c$, $F^{(k)}(u) \leftarrow 1$, $u.parent \leftarrow v$
11: Update($Q^{(k)}$)
12: **end if**
13: **end for**
14: **end while**
15: BackTrack Path
16: **end for**

In each round of A* search, we extract the state with the minimum f value from the open list, expand its outer neighbors and check for duplication. In general, Batched maze routing kernel is executed on a large batch of graphs in the detailed routing stage. There are almost 1,000 10,000 graphs (nets) in one batch without data dependency between any two graphs, providing a powerful condition for parallelization operation on FPGA to achieve efficient acceleration.

However, the typical maze routing algorithm should be modified to adapt the property of FPGA,as Algorithm 2 shows. The grid map is initialized in the memory and the start and end nodes are sent to the design so it can start solving the map. The priority queue Q, the temporary cost set T, and the vertices' state set F used in the hardware-optimized maze routing algorithm are also implemented on hardware. At the outer level, multiple routing graphs are processed concurrently, exploiting batch-level parallelism with thousands of nets in BRAM. At the inner level, for each graph, neighbour expansions across multiple directions are executed in parallel to accelerate wavefront propagation. This fine-grained directional

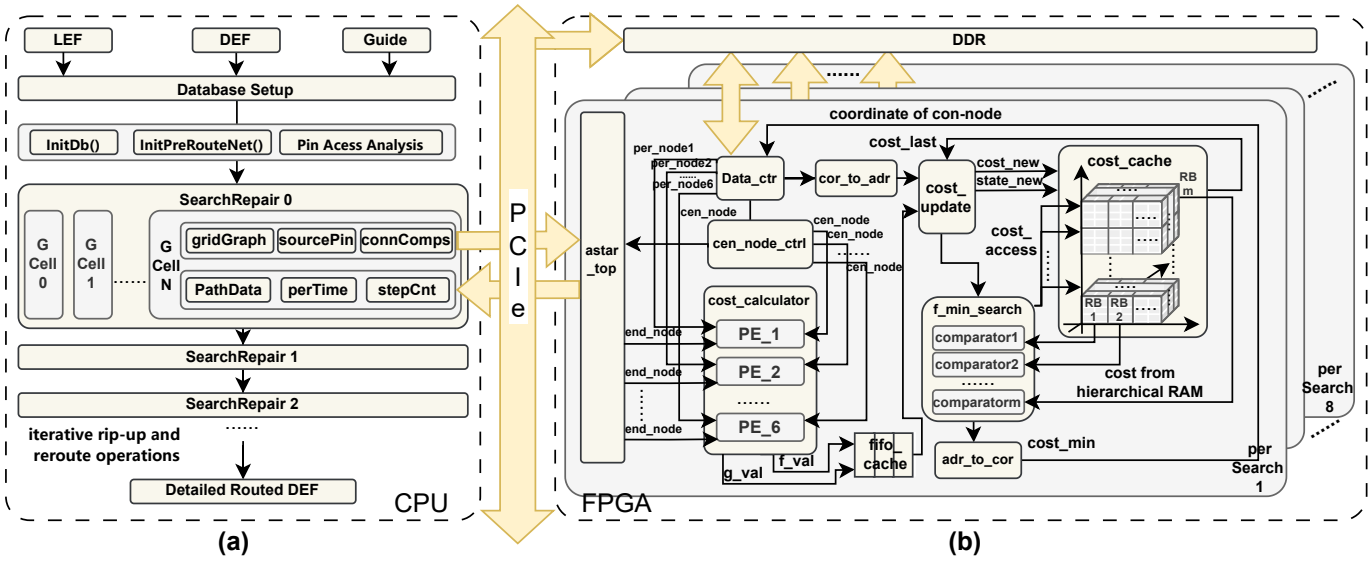


Fig. 2. The overview structure.(a) The flow of detailed router on GPU.(b) The overview of FPGA acceleration framework.

parallelism, combined with hardware-managed cost updates and queue operations, enhances routing throughput on FPGA.

In the process of generating routing diagram, take TritonRoute[?] as an example, through the breadth-first search (BFS) path planning method of extending and updating wavefront, the wavefront is dynamically generated and explored in six directions, southeast, northwest and upper, lower, and lower, to gradually update the cost information of each node to realize the path between the target point and the starting point. It can also be deployed on FPGAs for acceleration through full parallelism and data processing optimization.

IV. Design of Acceleration Module

A. The Overview of FPGA Acceleration Framework

Conventional implementations of A* search usually use two lists to store the states during its expansion, i.e., the **open list** and the **closed list**. The closed list stores all the visited states, and is used to prevent unnecessary repeated expansion of the same state. This list is often implemented by a linked hash table to detect the duplicated nodes. The open list normally stores the states whose successors have not been fully explored yet. The open list uses a priority queue as its data structure, typically implemented by a binary heap. States in the open list are sorted according to the function value of heuristic function f (x).

A lot of work proves that memory access is the time bottleneck of the algorithm compared to the computation time, as Figure 1 shows. So our work is dedicated to propose a FPGA-based implementation to accelerate the detailed routers, uses efficient memory access management and parallelism to achieve better performance. Figure 2 shows the overview of FPGA acceleration framework.

B. Hierarchica RAM Blocks

Instantiate multiple ram groups to store node information on the corresponding dimension. Traditional FPGA-based path planning algorithms have significant bottlenecks in maintaining **open list** and **closed list**. The dynamic sorting mechanism requires the storage structure to be reordered after each search iteration. This frequent sorting operation will not only consume a lot of programmable logic resources, but also produce a processing delay that does not match the hardware architecture, which seriously restricts the real-time performance of the system. In addition, dynamic memory management of variable-length data structures leads to fragmented access to Block RAM, resulting in reduced hardware resource utilization.

To solve this problem we observed, the dimensional organization of RAM can be efficiently managed through hierarchical mapping, in FPGA-based storage architecture design, as fig1 shows. For 2D data structure, row-priority storage strategy can be adopted, so that each RAM module can store the data node corresponding to the row (first dimension) independently. When extended to a three-dimensional structure, each RAM module can be responsible for storing node information in a fixed plane, such as the third dimension. Through the introduction of address mapping module, high-dimensional addresses (such as three-dimensional coordinates (x,y,z)) can be converted into two-dimensional storage addresses inside each RAM module. This hierarchical mapping mechanism significantly improves storage access efficiency and data throughput.

In addition, we offer a flexible dimension division architecture. The RAM storage can be divided into X, Y, and Z dimensions based on specific data characteristics, and can be adjusted according to the actual wiring situation, thereby achieving better acceleration effects. We initially use simple methods such as the y-z plane for partitioning, but the number of calculation periods is extremely large. This is because

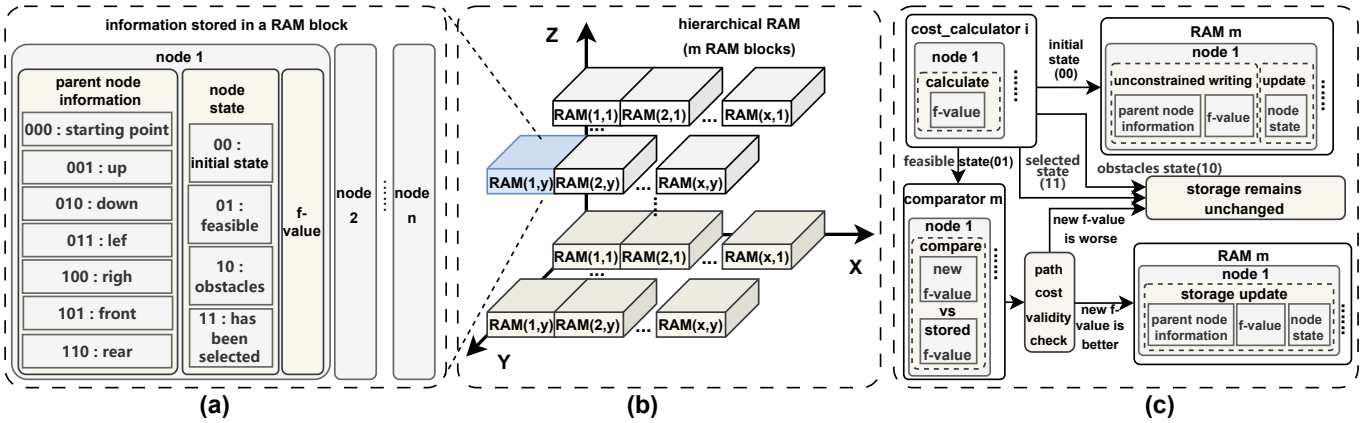


Fig. 3. RAM data structure on FPGA.(a)Information stored in a RAM block .(b)hierarchical RAM(a total of n RAM blocks).(c)The f-value update logic.

shuttling through the y-z plane frequently occurs during wiring. Therefore, we have made the following improvements: 1) Change the RAM division in the y-z plane direction to the x-y direction, and change the shuttle in the y-z plane to x-y to reduce the number of points inside the RAM; 2) Further meshes are carried out within the x-y plane to reduce the jurisdiction area of each RAM and further compress the maximum period of the minimum value search within each RAM. The above two points can effectively reduce the overall path search time.

By introducing the address mapping module, high-dimensional addresses (such as three-dimensional coordinates (x,y,z)) can be converted into two-dimensional storage addresses within each RAM module. This hierarchical mapping mechanism significantly improves the efficiency of storage access and data throughput. The address mapping logic is as follows:

As the known coordinate values is (x,y,z), we define:

$X_{\text{BLOCK_WD}}$: the number of data in x-dimension of RAM

$Y_{\text{BLOCK_WD}}$: the number of data in y-dimension of RAM

$Z_{\text{BLOCK_WD}}$: the number of data in z-dimension of RAM

$$x_{\text{f_ram_sel_cnt}} = \frac{\text{per_x}}{X_{\text{BLOCK_WD}}}$$
$$y_{\text{f_ram_sel_cnt}} = \frac{\text{per_y}}{Y_{\text{BLOCK_WD}}}$$
$$z_{\text{f_ram_sel_cnt}} = \frac{\text{per_z}}{Z_{\text{BLOCK_WD}}}$$

The RAM enable signal is:

$$f_{\text{ram_sel}} = x_{\text{f_ram_sel_cnt}} + y_{\text{f_ram_sel_cnt}} \times X_{\text{RAM_NUM}} + z_{\text{f_ram_sel_cnt}} \times X_{\text{RAM_NUM}} \times Y_{\text{RAM_NUM}}$$

The address of RAM Blocks on-chip is:

$$f_{\text{ram_sel}} = x_{\text{f_ram_sel_cnt}} + y_{\text{f_ram_sel_cnt}} \times X_{\text{RAM_NUM}} + z_{\text{f_ram_sel_cnt}} \times X_{\text{RAM_NUM}} \times Y_{\text{RAM_NUM}}$$

C. Data Structure and Update Control

After the path cost calculation module completes the f-value generation of the adjacent nodes of the central node, the system executes the storage update process based on logical control. First, Cost_cache memory is accessed through the coordinate addressing mechanism to extract the status flag bit and historical f-value of the target node: If the flag bit is detected as the initial state (00), the new f-value and status flag are directly written without constraints; If the status flag is explored state (01), the hardware comparator is triggered to perform the numerical relationship determination of the new f-value and the stored f-value in parallel, and the path cost validity check is performed at the same time to avoid calculation anomalies. Finally, the storage unit is selectively updated according to the judgment result: for the initial state node to load new data completely, the explored state node only performs overwrite storage and synchronously updates the state when the new f-value is better. This mechanism uses the state-driven conditional write strategy to reduce the redundant access to memory and data overwrite operations while maintaining the principle of path planning optimality.

D. Calculate Cost

Cost_calculator module calculates the f-values of 6 peripheral points in parallel according to the input center point and surrounding point information. The information in 6 directions is calculated in parallel to the Manhattan distance, and the F value of the 6 surrounding points is calculated based on F=G+H.

Cost_calculator module

E. Compare and Sorting Minimum F-Value

In the process of selecting the least cost node of distributed storage architecture, the system uses the hierarchical comparison mechanism of self-updating to realize the dynamic update of the center point. The built-in hardware minimum value tracking module of each storage unit immediately triggers the parallel comparison circuit in the unit when a new f-value is detected: real-time comparison of the currently stored

TABLE I
COMPARISON OF PERFORMANCE BETWEEN TRITONROUTE [?] DR.CU [?] AND OUR FPGA ACCELERATION TO COMPLETE BATCHED MAZE ROUTING KERNEL WITH ISPD 2018 [?] CONTEST BENCHMARKS.

ispd18	ISPD' 18 quality score					Routing Kernel(8 kernels)							
						Runtime(s)					Speed-up		
	TR	CU	RA-TR	[1]-CU	CoRA-TR	TR	CU	RA-TR	[1]-CU	CoRA-TR	RA-TR	[1]-CU	CoRA-TR
testl						4.41		2.03			2.2x		
test2						58.86		25.93			2.3x		
test3						110.07		52.4			2.1x		
test4						244.48		211.68			2.3x		
test5						109.42		40.39			2.3x		
test6						165.39		77.33			2.1x		
test7						246.12		111.98			2.2x		
test8						256.73		125.29			2.1x		
test9						264.21		128.19			2.1x		
test10						380.69		180.455			2.1x		

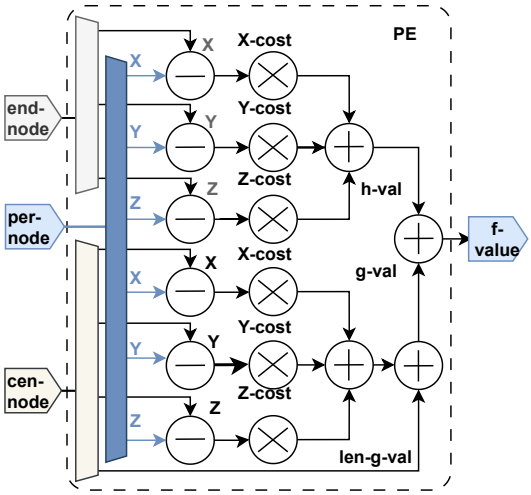


Fig. 4. The PE of Cost_Calculator module.

minimum f-value and the new written value through the special register group, dynamic update of the minimum cost value and its three-dimensional coordinate information, and generate an effective status flag bit to synchronize to the arbitration controller. When the valid flag bits of all storage units are ready, the hierarchical comparison architecture starts the global arbitration in stages. First, the local minimum values in each storage row are screened in parallel by row-level comparator array, and then the row-level minimum values are progressively compared by a tree arbiter based on depth-first strategy. Finally, the coordinates and path cost values of the globally optimal nodes are output.

After the selected global minimum is verified, the system loads the new central point information into the path planning logic through the three-dimensional coordinate bus. At the same time, double buffer register group is used to realize the decoupling of the computing unit and the storage unit to ensure the continuous operation of the comparison pipeline during the data update cycle. By means of distributed priority arbitration mechanism implemented by hardware, the architecture transforms the ergodic search in traditional algorithms into multilevel pipeline operation, which avoids the resource competition problem of centralized comparison architecture while maintaining real-time path planning.

V. EXPERIMENTAL RESULTS

A. Experimental Setup

Our acceleration system is implemented on a linux system with 8(?) cores Intel(R) Xeon(R) Gold 5218R CPU @ 2.10GHz with 64G Memory, and a Xilinx XC7VX690T(xc7vx690tffg1761-2) FPGA with 433.2K LUTs, 866.4k FlipFlops, 1.47K Block RAMs. The hardware design is synthesized by using Vivado 2020.2 with a targeted frequency of 125MHz.

B. Acceleration Efficiency Comparison

We use the classic detailed wiring test benchmarks of ISPD2018 and ISPD2019 to compare the performance between the following related works and justify the acceleration efficiency of our system. CU: Time was run on our hardware RA-CU: TR generally supports 8-thread CPU core parallelism. Correspondingly, we also use 8-thread parallel execution in hardware. When the FPGA frequency is much lower than that of the CPU, under the same degree of parallelism, we have achieved significant acceleration. Co-RA: WXL is adjusted with the parameter of search repair to make the number of iterations similar to DR CU, but in fact, it is still more than DR CU. [1] -CU: It is not open source, and the running time may vary on different hardware. For fairness, we compare the speedup ratio. They work on their platform in the same working environment as CU, and we work in the same working environment as CU running on our platform. The acceleration multiplies of the two groups can eliminate the interference of CPU factors and are more reasonable for comparing the acceleration capabilities. And they work with 12 threads. The number of threads is more than ours, but we are still faster than them. Table I shows the comparison of performance between TritonRoute [?] Dr.CU [?] and our FPGA acceleration to complete batched maze routing kernel with ISPD 2018 [?] contest benchmarks

TABLE II
CONSUMPTION OF HARDWARE RESOURCE UTILIZATION.

Resource	Utilization	Available	Utilization %
LUT	48334	433200	11.16
LUTRAM	7760	174200	4.45
FF	49145	866400	5.67
BRAM	96.50	1470	6.56
IO	124	850	14.59
GT	8	36	22.22
BUFG	10	32	31.25
MMCM	4	20	20.00
PLL	1	20	5.00
PCIe	1	3	33.33

C. Design Space Exploration of the RAM Block.

D. Consumption of Hardware Resource.

Table II shows Consumption of Hardware Resource Utilization.

VI. CONCLUSIONS

TABLE III
TABLE TYPE STYLES

Table Head	**Table Column Head**		
	Table column subhead	***Subhead***	***Subhead***
copy	More table copy[a]		

[a]Sample of a Table footnote.

ACKNOWLEDGMENT

The preferred spelling of the word "acknowledgment" in America is without an "e" after the "g". Avoid the stilted expression "one of us (R. B. G.) thanks ...". Instead, try "R. B. G. thanks...". Put sponsor acknowledgments in the unnumbered footnote on the first page.

REFERENCES

[1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," Phil. Trans. Roy. Soc. London, vol. A247, pp. 529–551, April 1955.

[2] J. Clerk Maxwell, A Treatise on Electricity and Magnetism, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp.68–73.

[3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in Magnetism, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.

[4] K. Elissa, "Title of paper if known," unpublished.

[5] R. Nicole, "Title of paper with only first word capitalized," J. Name Stand. Abbrev., in press.

[6] Y. Yorozu, M. Hirano, K. Oka, and Y. Tagawa, "Electron spectroscopy studies on magneto-optical media and plastic substrate interface," IEEE Transl. J. Magn. Japan, vol. 2, pp. 740–741, August 1987 [Digests 9th Annual Conf. Magnetics Japan, p. 301, 1982].

[7] M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.

IEEE conference templates contain guidance text for composing and formatting conference papers. Please ensure that all template text is removed from your conference paper prior to submission to the conference. Failure to remove the template text from your paper may result in your paper not being published.